

Job Sequencing and Tool Switching

Structural results and a branch-and-Benders-cut study

Shankar Narayanan

Advisors: Prof. Rafael Colares Prof. Annegret Wagler
Academic supervisor: Dr. Renaud Chicoisne

Laboratory of Informatics, Modelling and Optimisation of Systems (LIMOS)
ISIMA, Clermont Auvergne INP

Internship defence 2 September 2026



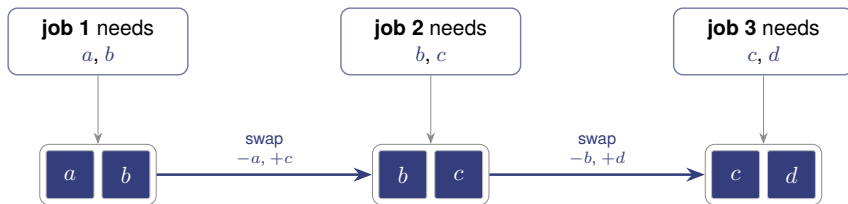
- 1 The problem
- 2 Structural results
- 3 A branch-and-Benders-cut algorithm
- 4 Strengthening the solver
- 5 Computational results
- 6 Limitations and conclusions

PART 1 OF 6

The problem

What the machine actually does, why the job order changes the cost, and the two questions this work answers.

A machine, a magazine, and a set of jobs



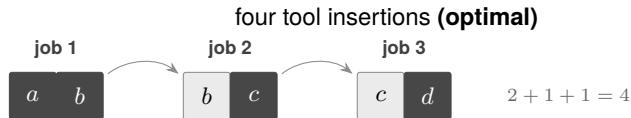
the magazine (the grey box) holds only two tools at a time

- One machine makes many different parts. Each part — a **job** — needs its own small set of tools.
- The tools sit in a **magazine** that holds only a few at a time. To run a job, every tool it needs must be in the magazine.
- Swapping a tool stops the machine, and each stop costs time. So the thing to minimise is the **number of tool insertions**.

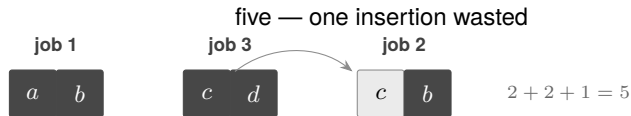
Two decisions: **the order of the jobs**, and **what stays loaded**.

Why the order matters

order 1, 2, 3



order 1, 3, 2



inserted here (counted)



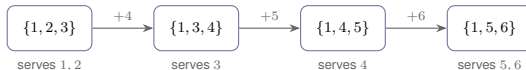
already loaded (free)

Same jobs, same magazine, same loading rule. Only the order changed — and running job 2 last forces a tool out and then back in.

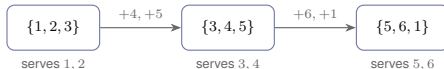
That second insertion is a **reinsertion**, and it is what this whole talk is about.

Schedules as walks through configurations

four configurations: cost 3



three configurations: cost 4

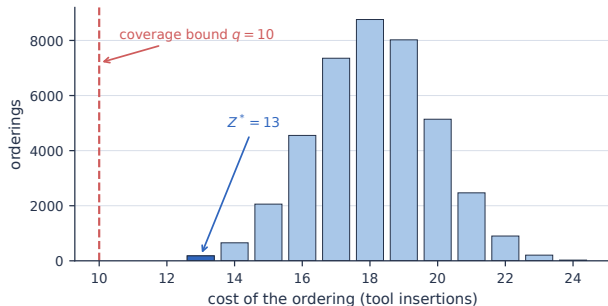


A **configuration** is one full magazine. A schedule is a walk from one to the next, costing the tools that enter along the way. This picture is behind both halves of the talk.

Two quantities, for the rest of the talk

Write b for the magazine capacity, T_j for the tools job j needs, and U for the tools the instance actually uses. Let Z^* be the fewest insertions any schedule achieves. At most b tools are loaded before the first job and every other one must enter at least once, so $Z^* \geq q := |U| - b$ — an elementary count.

One decision is easy, the other is not



40,320

orders of L1-1, a standard eight-job
benchmark

180

of them are optimal

Once the order is fixed

When a tool must enter a full magazine, remove the one whose next use lies furthest ahead — the *Keep Tool Needed Soonest* rule (KTNS). It is optimal, and it is fast.

Tang & Denardo (1988)

Choosing the order

NP-hard for every magazine capacity of two or more.

Crama et al. (1994)

$q = 10$ **vs** $Z^* = 13$

the count is three short

What practice does

Pack jobs into the fewest batches that fit the magazine, then order the batches. Known to be a b -approximation, and it does well in the one industrial study.

Crama & van de Klundert (1999); Burger et al. (2015)

Missing: an instance-sensitive account of when this construction loses.

What exact models do

Four decades of integer models. Of the three reimplemented here, the strongest relaxes to exactly the elementary count q — no better than counting.

Laporte et al. (2004); Catanzaro et al. (2015); da Silva et al. (2021)

Missing: a useful certificate when q is not the optimum.

1. Grouping

What does insisting on the *fewest* groups cost, once the order it produces is reloaded optimally?

Answered by: identities, zero-gap cases, a ratio bound, and a family where the loss grows without bound.

2. Exact optimisation

Can an exact loading oracle carry a solver past the counting bound q ?

Answered by: a branch-and-Benders-cut solver, four strengthenings, and a campaign of sixteen thousand runs.

Both live on the same walk: one restricts which walks are allowed, the other asks what can be *proved* about all of them.

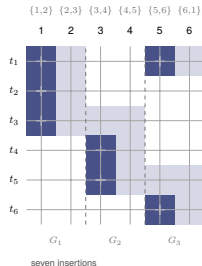
PART 2 OF 6

Structural results

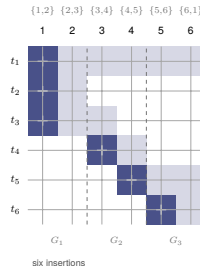
The heuristic industry uses is better than its published worst cases suggest — and worse than they can prove.

The standard worst-case example is not a worst case

(a) the walk: one magazine per group



(b) KTNS on the same job order



(a) What the literature draws

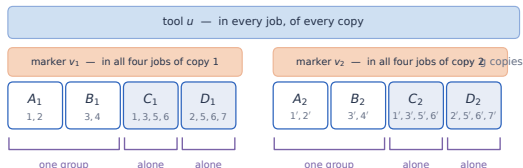
One magazine per group, held while that group runs. Every published worst case for this method is built on this picture. Here it pays **one extra insertion**.

(b) What the method actually does

It reloads the whole job order optimally at the end. On the same instance it pays **nothing extra** — it is optimal.

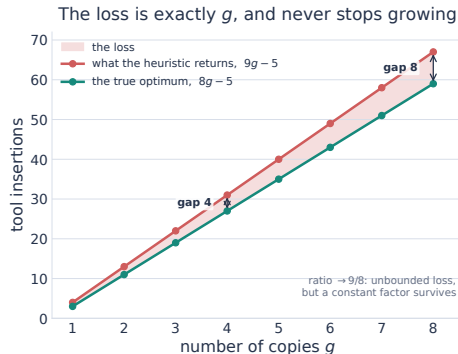
This is *the* example the field reaches for to show this heuristic losing. Under the method as specified, **it does not lose**.

So the loss is real — but you need a different construction



C_i and D_i already fill all six slots, so each must sit alone.
 A_i and B_i together need exactly six — but A_i with any job of another copy needs seven. The minimum grouping is forced and unique.

Every job shares one tool, so the instance is **connected** — this is not g separate problems placed side by side. The loss is exactly g and grows without bound.



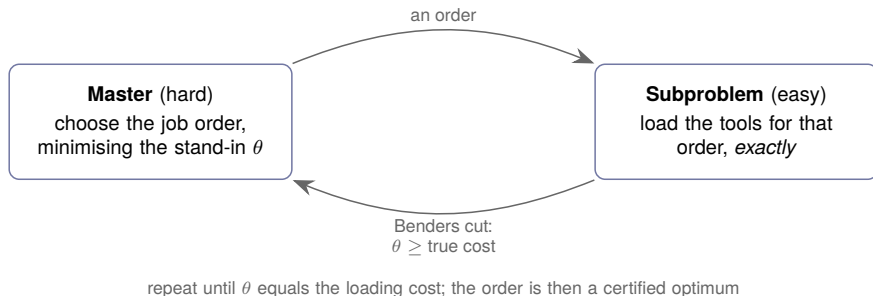
And the loss is exactly the price of insisting on the *fewest* groups: allow every grouping and the construction becomes exact. Nothing else in the method is at fault.

PART 3 OF 6

A branch-and-Benders-cut algorithm

Hand the easy half to an exact oracle, and let the hard half be searched. Where the cuts come from, and what they can say.

The decomposition



- 1 The master proposes an order and an optimistic loading cost θ .
- 2 The subproblem returns the **exact** loading cost of that order.
- 3 A dual solution produces a cut valid for every order and tight at the priced one.

Logic-based and combinatorial Benders lineage: *Hooker & Ottosson (2003)*; *Codato & Fischetti (2006)*.

The master problem

Let d be an empty depot and $x_{ij} = 1$ when j follows i .

$$\min \theta, \quad \sum_{i \neq j} x_{ij} = 1 \ (\forall j), \quad \sum_{j \neq i} x_{ij} = 1 \ (\forall i),$$

$$\sum_{i,j \in S} x_{ij} \leq |S| - 1 \quad (S \subseteq J).$$

Convention: theory uses free-initial $q = |U| - b$; the BBC oracle charges an empty start, so $\theta \geq |U|$.

Three bound rows, before any cut is generated:

$$\underbrace{\theta \geq |U|}_{\text{coverage}} \quad \underbrace{\theta \geq \sum_{i,j \in J} w_{ij} x_{ij} + \sum_{j \in J} |T_j| x_{dj}}_{\text{pairwise, } w_{ij} = \max(0, |T_i \cup T_j| - b)} \quad \underbrace{\theta \geq w_{ijk}(x_{ij} + x_{jk} - 1)}_{\text{triplet, optional}}$$

All three are visible to the root relaxation. Keep that in mind — the last part of the talk is about how little they turn out to be worth.

The subproblem: loading for a fixed order

Fix $\bar{x}$. Let $y_{j,t}$ indicate that tool t is loaded at job j , and let $z_{j,t}$ charge its insertion.

$$\begin{aligned} \min \quad & \sum_{j,t} z_{j,t} \\ \text{s.t.} \quad & y_{j,t} \geq 1 && t \in T_j && (\nu_{jt}) \\ & \sum_t y_{j,t} \leq b && \forall j && (\mu_j) \\ & z_{j,t} \geq y_{j,t} - y_{i,t} - (1 - \bar{x}_{ij}) && \forall i \neq j, t && (\lambda_{ijt}) \\ & z_{j,t} \geq y_{j,t} - (1 - \bar{x}_{dj}) && \forall j, t && (\lambda_{jt}^d). \end{aligned}$$

The last row charges the first job because the machine starts empty.

Exact oracle

Once $\bar{x}$ fixes a path, the remaining y, z matrix has the consecutive-ones property and is totally unimodular. Therefore

$$\min\{\text{loading LP at } \bar{x}\} = \text{KTNS}(\bar{x}).$$

Primal meaning

Every incumbent order is evaluated at its true loading cost.

Remaining difficulty

The master must still certify that no better order exists.

The oracle uses the fixed-order result of *Tang & Denardo (1988)*; it is not a heuristic price.

For an optimal dual solution $(\bar{\lambda}, \bar{\lambda}^d, \bar{\mu}, \bar{\nu})$:

$$\theta \geq \sum_{i \neq j} \sum_t \bar{\lambda}_{ijt} (x_{ij} - 1) + \sum_j \sum_t \bar{\lambda}_{jt}^d (x_{dj} - 1) - b \sum_j \bar{\mu}_j + \sum_j \sum_{t \in T_j} \bar{\nu}_{jt}$$

Global

Dual feasibility does not depend on the order.

Exact locally

At $\bar{x}$, the cut equals $\text{KTNS}(\bar{x})$.

Fractional-ready

It is affine in x and can be separated before integrality.

Initial defect

Omitting $\sum_{j,t} \bar{\lambda}_{jt}^d (x_{dj} - 1)$ removes a non-positive correction. The cut can then overestimate orders with other first jobs and remove the true optimum.

Detection

Independent brute-force testing found **193** violated optimal points.

Correction

Restore the depot term, rederive the cut from the implementation, rerun the complete campaign.

This is why the final validation uses independent enumeration and dynamic programming, not only agreement with the solver.

PART 4 OF 6

Strengthening the solver

Four standard accelerations. One aims at the incumbent, three at the bound. Only one of them helps.

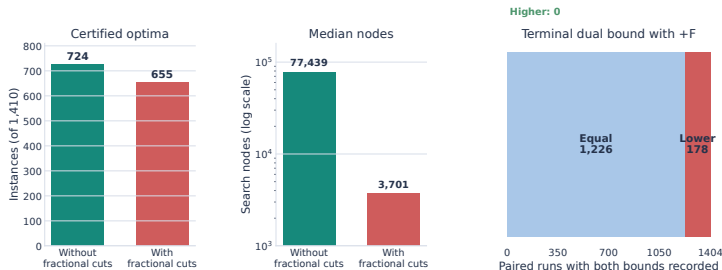
Mechanism	Target	Implementation here
Fractional separation	dual bound	price the same cut at fractional x
Root HGS incumbent	primal bound	warm start and initial cut pool
Conflict-graph row	dual bound	lower-bound the number of groups
Pareto lifting	cut choice	select a deeper dual optimum

How they were tested. One switch at a time on a common base, then all three together, then the base with fractional separation removed — nine Benders regimes in all, each validated against brute force before the campaign. *Rahmaniani et al. (2017)*

Three different targets. **Watch which kind helps.**

Fractional Benders cuts

Method. Price the same affine dual cut at a fractional master solution, before an integral path exists.



Fractional separation generated 67.0 million cuts, and also disables presolve.

67 million

cuts generated

20×

fewer nodes explored

0 of 1,404

terminal bounds improved

The third number is the finding, and it is the one that is clean. It also loses 69 certified instances — but this regime *also* disables presolve, so that figure is confounded and I do not lean on it. **Whether these cuts ever raise the bound is not confounded: they never do.**

Compact HGS at the root *Mecler et al. (2021)*:

constructive seeds $\rightarrow$ VNS improvement
 $\rightarrow$ order crossover + diversity $\rightarrow$ exact KTNS scoring

Warm start

The best order supplies an incumbent before branching.

Initial cut pool

Its exact loading dual supplies a valid cut before the first relaxation.

Measured effect: +31 instances — the only strengthening that gains any

And it raises the obvious question. Removing fractional separation is worth +69; this is worth +31. **The two were never combined.** That regime is the first thing I would run next.

Conflict-graph row

Jobs that cannot share a magazine form edges in G . A colouring lower bound gives

$$\theta \geq (\chi_{lb} - 1) + \min(b, |U|).$$

It is useful only when grouping structure beats coverage.

Atamtürk et al. (2000)

Measured effect: -1 instance.

Pareto-optimal lifting

Among alternative optimal loading duals, maximise the cut value at an interior core point; retain the ordinary cut as well.

Magnanti & Wong (1981); Papadakos (2008)

Measured effect: -37 instances. A deeper cut in the same family is still arc-supported.

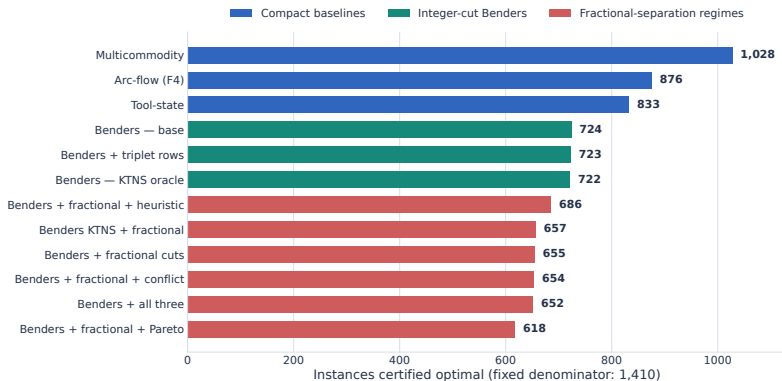
PART 5 OF 6

Computational results

Sixteen thousand runs under one protocol. The compact models win, and the reason is a single quantity.

How far each method gets

1,410 instances $\times$ 12 configurations, one hour each. Every non-optimal outcome counts as unsolved.



Every compact baseline beats every Benders regime on the full denominator. But half that denominator does not discriminate — and the next slide separates it.

Half the benchmark cannot tell the methods apart

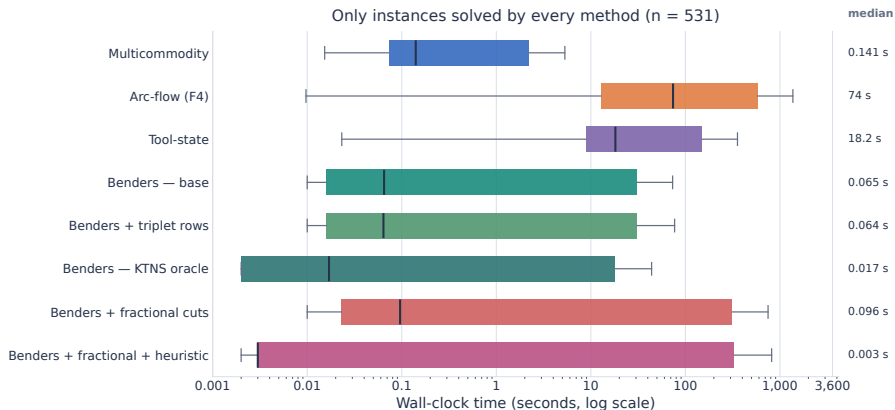
Laporte3 and Laporte4 are 670 of the 1,410 instances, and **all three compact models close 100% of both**. They are denominator, not evidence. On the 740 instances that do discriminate:

	certified	share
Multicommodity	358/740	48.4%
Benders + fractional + heuristic	213/740	28.8%
Arc-flow (F4)	206/740	27.8%
Benders — base	192/740	25.9%
Tool-state	163/740	22.0%

The honest comparison

On the half of the benchmark where the comparison means anything, this solver is **ahead of two of the three baselines** — while running on one solver thread against their four. Only the multicommodity model is clearly ahead.

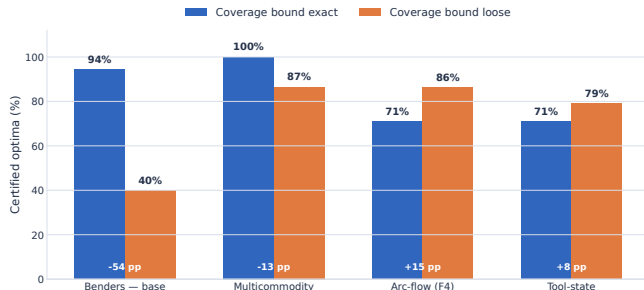
Solving times



On the 531 instances everyone solves, the picture inverts — the Benders medians sit orders of magnitude lower. But look at the tails.

It closes an instance at once, or not within the hour. That is the signature of a bound that is either exactly right or of no use.

What separates the two cases



94% **vs** 40%

closed when q is exact, against when it is
loose

87%

of roots are the coverage row and nothing
more

58%

of time-limited runs already hold the
optimum

But this is a fact about the method, not the instances. F4 moves the *other* way and closes every one of the loosest; the multicommodity model, whose relaxation is also q , barely moves at all. What collapses is a master carrying q and, at the root, little else.

PART 6 OF 6

Limitations and conclusions

Why the bound does not move, what this study does not show,
and where the next attempt should go.

The cost is not a local quantity

Recall the counting bound from the start: $q = |U| - b$ — one insertion for every used tool that could not already have been loaded.

Now write β_t for the number of separate stretches during which tool t sits in the magazine. Every insertion of t begins one, so

$$Z^* = q + \sum_{t \in U} (\beta_t - 1).$$

One stretch per tool gives exactly q . **Everything above q is a reinsertion** — a tool evicted and brought back.

A reinsertion happens when the jobs needing a tool are pushed apart by others whose requirements cannot sit alongside it — a property of the **global interleaving** of the schedule, not of which jobs are adjacent.

What the master actually carries

One row indexed by *arcs*, triplet rows that are the weak linearisation of a product, and a constant coverage row. Every optimality cut it generates is arc-supported too. So it learns only **adjacency-indexed** facts about a cost that is not adjacency-indexed.

If the diagnosis is right, the fix is an inequality that counts across a *stretch* of the schedule rather than across one adjacency. So I derived one.

The window inequality

Take any block of consecutive positions. Every tool needed somewhere inside that block is either already loaded when the block begins, or is inserted within it. Counting both sides bounds the insertions inside the block — over a **span**, not an arc.

It looked promising

Screened over integral schedules it had several times the headroom of the rows the solver already carries.

It delivered nothing

It moved **0 of 37** measured roots — and then the proof of why: the fractional row implementing it is *implied* by rows the formulation already has.

What this study does not show

- **The heuristic census is small.** One fixed census of 1,260 instances at three to five jobs. The connected family settles *existence* and unboundedness; it says nothing about how often such gaps occur at industrial sizes.
- **One hour, one machine, one collection — and F4, not F5.** Every solve count comes from a single limit on one node type, and the compact baseline is weaker than the best available. Both points run *against* the conclusion drawn here, not for it.
- **The best-looking combination was never run.** Every strengthening was tested on top of fractional separation. But removing fractional separation is worth +69 instances and the root heuristic is worth +31, so the obvious configuration — the heuristic *without* fractional cuts — is missing from the campaign. It should have been the tenth regime.

Established

- Grouping has distinct walk and KTNS-postprocessed values.
- Minimum cardinality is exactly the restriction behind the walk gap.
- The specified heuristic has an unbounded additive gap on a connected family.
- The tested Benders method usually finds good orders but often cannot certify them.
- On the discriminating half of the benchmark it is competitive with two of the three baselines, at a quarter of their solver threads.

The missing object is a useful fractional certificate for non-local tool returns.

Next tests

- Run the heuristic *without* fractional separation — the missing regime.
- Price tool residency over spans, not only adjacent arcs.
- Disaggregate θ by position and chain PTF configurations.
- Find the exact worst ratio, the strongest K^* -dependent bound, and when KTNS closes the walk gap.
- Test weighted tool costs and learned column selection.

Thank you

Questions welcome

Key references — the problem, the heuristic, the models

Loading a fixed order is solved in closed form, and the rule is optimal.

Tang, C. S. & Denardo, E. V. (1988). Models arising from a flexible manufacturing machine, Part I. *Operations Research* 36(5), 767–777.

Ordering is NP-hard for every capacity $b \geq 2$; the loading LP stays exact when tool costs differ, which is why the solver extends to that case.

Crama, Y., Kolen, A. W. J., Oerlemans, A. G. & Spieksma, F. C. R. (1994). Minimizing the number of tool switches on a flexible machine. *IJFMS* 6, 33–54.

The two-phase construction industry uses is a b -approximation — the result this report sharpens into an instance-sensitive theory — and it performs well in the one industrial study that measures it against exact optima.

Crama, Y. & van de Klundert, J. (1999). Worst-case performance of approximation algorithms for tool management. *Naval Research Logistics* 46, 763–783. Burger, A. P., Jacobs, C. G., van Vuuren, J. H. & Visagie, S. E. (2015). Scheduling multi-colour print jobs with sequence-dependent setup times. *J. Scheduling* 18, 131–145.

The three compact baselines reimplemented here: the tool-state model, the strongest compact arc family and the source of the counting bound q , and the multicommodity model whose relaxation is also q yet which loses only thirteen points.

Laporte, G., Salazar-González, J. J. & Semet, F. (2004). Exact algorithms for the job sequencing and tool switching problem. *IIE Transactions* 36, 37–45. Catanzaro, D., Gouveia, L. & Labbé, M. (2015). Improved ILP formulations for the SSP. *EJOR* 244, 766–777. da Silva, T. T., Chaves, A. A. & Yanasse, H. H. (2021). A new multicommodity flow model for the SSP. *IJPR* 59, 3617–3632.

Key references — the decomposition, and the backup material

Benders cuts over a combinatorial subproblem — the lineage this solver belongs to, and the cut form it uses.

Hooker, J. N. & Ottosson, G. (2003). Logic-based Benders decomposition. *Mathematical Programming* 96, 33–60. Codato, G. & Fischetti, M. (2006). Combinatorial Benders' cuts for MILP. *Operations Research* 54(4), 756–766.

The survey all four strengthenings come from, including the initial cut pool.

Rahmaniani, R., Crainic, T. G., Gendreau, M. & Rei, W. (2017). The Benders decomposition algorithm: a literature review. *EJOR* 259, 801–817.

The deepest cut among alternative dual optima, and the free core point that makes it practical — the form implemented here.

Magnanti, T. L. & Wong, R. T. (1981). Accelerating Benders decomposition. *Operations Research* 29(3), 464–484. Papadakos, N. (2008). Practical enhancements to the Magnanti–Wong method. *ORL* 36, 444–449.

The conflict-graph inequalities, and the hybrid genetic search used as the root incumbent — the only strengthening that gained instances.

Atamtürk, A., Nemhauser, G. L. & Savelsbergh, M. W. P. (2000). Conflict graphs in solving integer programming problems. *EJOR* 121, 40–55. Mecler, J., Subramanian, A. & Vidal, T. (2021). A simple and effective hybrid genetic search for the SSP. *C&OR* 127, 105132.

Backup material: what the PCF pricing problem is, dual stabilisation, the cut-selection framework the learned prototype follows, and where the construction generalises.

Goldschmidt, O., Nehme, D. & Yu, G. (1994). Note on the set-union knapsack problem. *NRL* 41, 833–842. Crama, Y. & Oerlemans, A. G. (1994). A column generation approach to job grouping. *EJOR* 78, 58–80. Lübbecke, M. E. & Desrosiers, J. (2005). Selected topics in column generation. *Operations Research* 53(6), 1007–1023. Tang, Y., Agrawal, S. & Faenza, Y. (2020). Reinforcement learning for integer programming: learning to cut. *ICML*. Pop, P. C., Cosma, O., Sabo, C. & Pop Sitar, C. (2024). A comprehensive survey on the generalized traveling salesman problem. *EJOR* 314, 819–835.

Backup slides

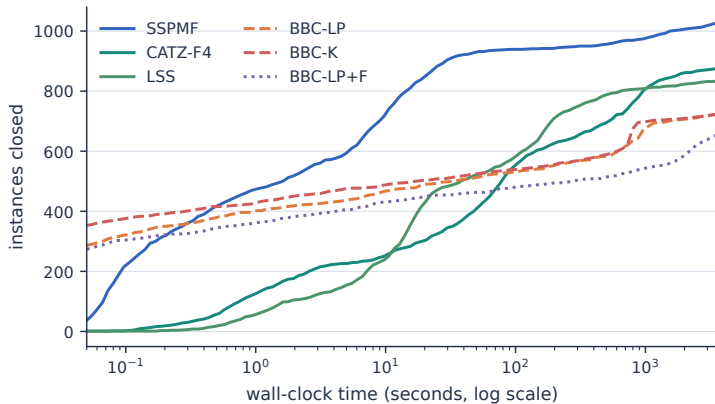
Supplementary evidence and alternative paths

Configuration	Catanzaro	Crama	Laporte3	Laporte4	Laporte5	Laporte7	All
SSPMF	62/160	68/160	340/340	330/330	162/340	66/80	1028/1410
CATZ-F4	44/160	44/160	340/340	330/330	74/340	44/80	876/1410
LSS	41/160	42/160	340/340	330/330	39/340	41/80	833/1410
BBC-LP	34/160	35/160	340/340	192/330	84/340	39/80	724/1410
BBC-LP+T	34/160	34/160	340/340	192/330	84/340	39/80	723/1410
BBC-K	35/160	35/160	340/340	190/330	83/340	39/80	722/1410
BBC-LP+F+H	39/160	37/160	316/340	157/330	97/340	40/80	686/1410
BBC-K+F	33/160	31/160	316/340	156/330	86/340	35/80	657/1410
BBC-LP+F	33/160	32/160	313/340	156/330	85/340	36/80	655/1410
BBC-LP+F+C	33/160	32/160	311/340	157/330	85/340	36/80	654/1410
BBC-LP+ACC	39/160	37/160	286/340	154/330	97/340	39/80	652/1410
BBC-LP+F+P	31/160	32/160	282/340	155/330	83/340	35/80	618/1410

The families *separate* the methods rather than rank them uniformly. Laporte4 splits them sharply — all 330 for every compact model, between 154 and 192 for Benders. And every configuration carrying +F sits below all three that do not separate at fractional points.

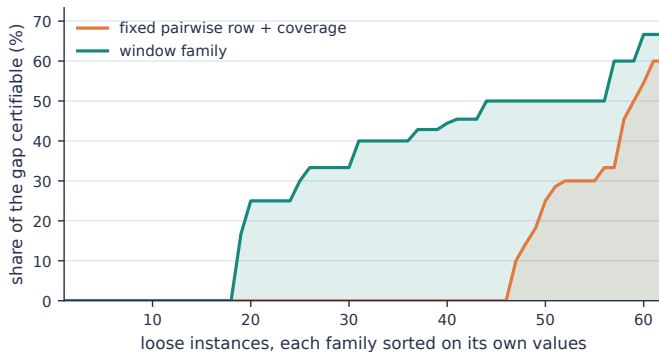
- **No two methods disagree.** 988 instances were closed by at least two methods, giving 41,514 method pairs that both proved an optimum. In none of them do the two values differ.
- **A fixed, audited denominator.** Every planned canonical pair counts once, certified or not; any non-optimal outcome counts as not closed. 16,920 planned outcomes, all present.
- **The identity of a run was tightened.** The historical runner keyed on file name and configuration alone, so repeated Crama names left 58 planned pairs unrecorded; they were completed under the same protocol before aggregation. The key is now (family, file, jobs, tools, capacity, configuration).
- **A predicted identity, confirmed on real data.** Empty-start and free-initial costs must differ by the initial load. SSPMF reports the latter; the difference equals b on 1,019 of the 1,028 instances it closed — and is smaller on exactly the nine whose optimum opens with a job needing $< b$ tools.
- **Every figure is regenerated from the raw ledger** by a checking script, and 58 independent mathematical checks re-run from the source data.

Backup — would a longer time limit change the story?



The counts would move, and they would not move equally for all methods — which is why the comparison is reported at one fixed limit and the conclusions rest on relaxation values rather than on solve counts.

Backup — the ceiling measurement, in full



Enumerating *every* sequence of the 62 loose cases in a fixed 81-instance sample at eight jobs gives the largest value each family could take on an integral schedule. The window family has roughly three and a half times the headroom of the pairwise rows.

That is a ceiling, not a bound — and the fractional row that implements it was then proved redundant. The gap between those two sentences is the honest result of this part of the work.

The question. At a node the master has several admissible cuts: one per candidate order evaluated, the coverage row, and one per violated fractional point. A fixed rule decides. Would a learned policy do better?

A prototype was built following the cut-selection framework of (*Tang et al., 2020*): selection as a Markov decision process — state is the relaxation point plus a candidate pool summarised by violation, support size, right-hand side and mean coefficient; reward is the rise in the dual bound; the policy is a linear softmax trained by REINFORCE.

Trained and evaluated on **knapsack cover cuts**, not on the SSP: the Benders solver needs a commercial solver the development environment did not have. 2,000 episodes at each of six sizes, one seed each, held-out evaluation.

n	learned	random	gain
10	1.3855	0.8734	+58.6%
20	0.7482	0.5021	+49.0%
26	0.5661	0.3279	+72.7%

Two things follow and only two: **the machinery learns**, and that is a statement **about knapsack cover cuts**. An appendix, not a method.

The reason is the same one the whole talk turns on

Cut selection chooses *among cuts a method can already generate*. Its established role in the literature is to pick well, not to make a family stronger.

In this campaign the available selections did not improve the terminal bound on a single instance — 67 million fractional cuts raised it on none. A selector cannot select a strong cut that does not exist.

Where learning would be worth trying instead. Column selection for the position-indexed pricers, whose coupled-pair oracle is the measured bottleneck. The non-learned version of the same idea is automatic dual stabilisation.

The obstacle

Among the three reimplemented compact baselines, even the strongest relaxation equals $q = |U| - b$; the other two are generally below it. The campaign therefore compares closure when q is exact and when it is loose.

Reformulating over **magazine configurations** attacks this from the other side. A configuration is a set $C \subseteq U$ with $|C| = b$; these become the columns, generated on demand. Three formulations were built.

	Column	Linear relaxation
PCF	configuration at a position	$= q$ with the counting rows, 0 without
PCF'	prefix form of PCF	$= q$, symmetry removed
PTF	<i>transition</i> between two	$\geq q$, and can exceed it

$y_C^k = 1$ if configuration C occupies position k ; $w_t^k \geq 0$ counts an insertion of t at position k .

$$\min \sum_{t,k} w_t^k$$

$$(P) \quad \sum_C y_C^k = 1, \quad k = 1, \dots, n$$

$$(Cov) \quad \sum_k \sum_{C \supseteq T_j} y_C^k \geq 1, \quad j \in J$$

$$(W) \quad w_t^k \geq \sum_{C \ni t} (y_C^k - y_C^{k-1}), \quad t \in T, k \geq 2$$

$$(T) \quad \sum_{k \geq 2} w_t^k + \sum_{C \ni t} y_C^1 \geq 1, \quad t \in U$$

The counting rows are load-bearing

Without (T) the relaxation is **zero** — one convex blend of configurations, repeated at every position, satisfies (P), (Cov) and (W) at no cost. With (T) it is exactly q . *Proved, and confirmed on all 2,108 recorded roots.*

$z_{C,C'}^k \geq 0$ is a column for the step from position k to $k + 1$ leaving C and entering C' , at cost $d(C, C') = |C' \setminus C|$.

$$\min \sum_k \sum_{(C,C')} d(C, C') z_{C,C'}^k$$

$$(S) \quad \sum_{(C,C')} z_{C,C'}^k = 1, \quad k = 1, \dots, n - 1$$

$$(F) \quad \sum_{(C,C'): t \in C'} z_{C,C'}^k = \sum_{(D,D'): t \in D} z_{D,D'}^{k+1}, \quad t \in T$$

(Cov) every job served at the head or tail of some step

$$(T) \quad \sum_k \sum_{(C,C'): t \notin C, t \in C'} z_{C,C'}^k + \sum_{(C,C'): t \in C} z_{C,C'}^1 \geq 1, \quad t \in U$$

What transition columns add

The counting rows give $\text{LP}(\text{PTF}) \geq q$, while each column carries a transition between configurations. On **2 of 232** recorded roots the exact LP value was 6 rather than $q = 5$; this is an observation, not a family-wide guarantee.

The pricing problem

Choosing the entering configuration is **NP-hard**: pick b tools to collect per-tool duals plus a bonus for each job fully covered. PTF prices *coupled pairs*, so a node costs far more again.

Rows stay small

$O(n|T|)$ and fixed; only the columns are generated.

What was measured

- PCF' root = q on **all** 2,108 recorded runs — the proposition confirmed on the benchmark.
- PTF exceeds q on **2 of 232** roots, rising from 5 to 6 where the optima are 12 and 13.
- Nothing above nine jobs was closed: a Python pricer against compiled solvers.

Sound, and as it stands too rare and too small to close the certificate gap.